

1. Общее описание лабораторных работ

1.1. Общая цель лабораторных работ

При программировании трехмерной компьютерной графики используются следующие средства:

- язык C++, являющийся стандартом де-факто в этой области;
- программные интерфейсы для трехмерной графики OpenGL, Vulkan, Metal, WebGL;
- шейдерные языки, такие как:
 - ▶ GLSL (OpenGL shading language, [1,2]),
 - ▶ HLSL (DirectX High-Level Shader Language, [3]),
 - ▶ MSL (Metal Shading Language, [4]),
 - ▶ RSL (RenderMan Shading Language, [5]) и другие.

Здесь мы не затрагиваем область инженерной графики, которая имеет свою специфику.

Перечисленные языки и библиотеки довольно низкоуровневые и сложны для освоения и требуют наличия уверенных знаний основ программирования, понимания работы компьютера в целом и видеопроцессоров (видеокарты) в частности, а также хорошего знания математических основ компьютерной графики.

Так как предмет «компьютерная геометрия» изучает именно математические основы компьютерной графики, то основной упор делается на освоении математического аппарата, понимании математических абстракций и концепций, лежащих в основе компьютерной графики. Следовательно и лабораторные работы проверяют в первую очередь полноту освоения математического аппарата.

Для того, чтобы обучающиеся могли практически полностью сосредоточиться именно на математических аспектах и не «сражаться» с техническими трудностями низкоуровневого программирования, для выполнения лабораторных работ был выбран язык программирования Python в связке с библиотеками NumPy[6] и Matplotlib[7]. В редких случаях может понадобиться библиотека SciPy[8] и Quaternion[9].

1.2. Особенности использования Matplotlib

От учащихся все же также требуется грамотное использование языка Python и в особенности библиотеки Matplotlib, поэтому следует учесть следующие замечания.

- Все изображения должны быть читаемые, то есть иметь высокое разрешение (150–200dpi), в некоторых случаях отображать координатную сетку, подписи к осям координат.
- Следует знать и использовать не только одну лишь функцию `plot` для всего, а также и специализированные функции для отображения прямых `xvlines`, `hlines`, `axvline`, `axhline`, `axline`, многоугольников `patches.Polygon`, а также других примитивов из подмодуля `patches`.
- Только в одной лабораторной работе можно использовать трехмерные возможности Matplotlib (`projection='3d'`). В задании это оговаривается специально! Во всех остальных случаях допускается использование *только* двумерных средств. Наша цель в большинстве лабораторных работ — научиться самостоятельно строить двумерные проекции трехмерных объектов, а не пользоваться какими-то функциями или библиотеками, которые делают все вычисления за нас.
- Следует использовать объектно-ориентированный интерфейс Matplotlib, то есть вызывать **все** основные функции через объект Axes:

```
import matplotlib.pyplot as plt
fig = plt.figure()
ax = fig.subplots()
```

Подробности можно узнать из методического пособия, на странице документации [по ссылке](#), а также на лекциях и семинарах. В интернете есть огромное количество примеров, где используется устаревший программный интерфейс, который был сделан в стиле математического пакета Matlab и который в настоящее время считается устаревшим, не развивается, не поддерживается и не поддерживает некоторые функции и настройки. За использование старого интерфейса будут снижаться баллы.

- Формулы на рисунках должны иметь математический вид: x^2 вместо $x^{\wedge}2$, $\sin x$ вместо $\sin x$ (а в чем разница?), $\sqrt{x^2 + y^2}$, вместо, например, $\text{sqrt}(x^2+y^2)$ и т.д. Греческие буквы также должны отображаться корректно: ϵ , α , β , θ , вместо e , α , β , θ и т.д.
- Во многих случаях крайне важно соблюдать одинаковый масштаб осей координат! Без этого итоговое изображение может исказиться и не соответствовать действительности. Например, вместо куба вы увидите параллелепипед, вместо окружности эллипс, вместо прямого угла острый или тупой. Для настройки масштаба используется функция `ax.scale`.

1.3. Советы по выполнению лабораторных работ

- Теоретический материал прежде всего следует искать в презентациях и в книгах из списка литературы. Если не получается найти там, то скорее всего вы что-то не поняли или пропустили, поэтому на следующем шаге лучше всего спросить у преподавателя. Если вы решили все же искать в интернете, то нужно сохранить ссылку, где вы нашли ответ на свой вопрос и при защите лабораторной иметь возможность показать этот материал преподавателю.
- Каждое задание лабораторной работы разбито на несколько пунктов. В большинстве случаев все пункты можно реализовать в виде одной цельной программы, так как выполнение каждого последующего пункта добавляет к программе что-то новое, но не отменяет предыдущие пункты.
- Все задания лабораторных работ рекомендуется выполнять в Jupyter Notebook. В нем же можно записать любую дополнительную или вспомогательную информацию: формулировки заданий, формулы, определения, пояснения и т.д. Делается это в отдельных ячейках, в которых текст можно форматировать с помощью языка разметки Markdown с добавлением формул в разметке LaTeX.

1.4. Защита лабораторных работ

- Для защиты лабораторной работы преподавателю нужно показать Jupyter Notebook (проще говоря блокнот). Все ячейки в блокноте должны быть работоспособны то есть выполняться без ошибок.
- Нужно своими словами сформулировать основную суть каждого задания, запустить соответствующие ячейки, прокомментировать рисунки, которые в результате получились. Если в задании лабораторной работы есть какие-то вопросы, то на эти вопросы следует ответить заранее.
- Во время защиты можно пользоваться всеми материалами: презентациями, методичками, какими-то своими конспектами, дополнительными пояснениями, добавленными в блокнот.
- Основная цель защиты для преподавателя — убедиться, что учащийся усвоил теоретический материал и сумел его использовать для выполнения практического задания. И самое важное, что сделал это сам или, по крайней мере, разобрался в материале и программе.
- Также в какой-то мере оценивается умение формулировать мысли и складно и структурировано их излагать.

- Можно использовать ChatGPT или что-то подобное, при условии, что вы можете объяснить все, что он сгенерировал или можете проверить и как-то доказать, что сгенерированный код верен.

1.5. За что снижаются балы

- Невыполненные задания.
- Неспособность объяснить что делает тот или иной кусок кода.
- Неспособность дать определение того или иного используемого термина.
- За фразы вроде «Эту формулу я нашел в интернете, где не помню, когда не помню, показать не могу, объяснить не могу», так как они лишь заменяют «я взял готовое и выдаю за свое».
- Фраза «Я делал это давно (аж целую неделю тому вперед) и уже забыл» также заменяет более честную фразу «Про лабораторную я вспомнил за день за сдачи, раздобыл готовую и пытаюсь сдать».

1.6. За что можно получить дополнительные баллы

- За нахождение опечаток/ошибок в формулах в презентациях и формулировках лабораторных работ.
- За нахождение непонятных/невнятных мест в тексте презентаций, которые не получается понять при самостоятельно разборе. Сообщите о таких местах преподавателю, сформулируйте что вам не понятно.
- Исправление ошибок и нечетких формулировок в заданиях лабораторных работ.

Библиография

1. GLSL specs 4.6 [Электронный ресурс]. URL: <https://www.khronos.org/registry/OpenGL/specs/gl/GLSLangSpec.4.60.pdf> (дата обращения: 17.10.2024).
2. GLSL github repository [Электронный ресурс]. URL: <https://github.com/KhronosGroup/glsl> (дата обращения: 17.10.2024).
3. HLSL Specifications [Электронный ресурс]. URL: <https://github.com/microsoft/hlsl-specs> (дата обращения: 17.10.2024).
4. Metal Shading Language Specification [Электронный ресурс]. URL: <https://developer.apple.com/metal/Metal-Shading-Language-Specification.pdf> (дата обращения: 17.10.2024).
5. Pixar's RenderMan [Электронный ресурс]. URL: <https://renderman.pixar.com/> (дата обращения: 27.01.2025).
6. NumPy [Электронный ресурс]. URL: <https://numpy.org/> (дата обращения: 27.01.2025).
7. Matplotlib [Электронный ресурс]. URL: <https://matplotlib.org/> (дата обращения: 27.01.2025).
8. SciPy [Электронный ресурс]. URL: <https://www.scipy.org/> (дата обращения: 27.01.2025).
9. Quaternion [Электронный ресурс]. URL: <https://github.com/moble/quaternion> (дата обращения: 03.03.2025).
10. Савелов А.А. Плоские кривые. Систематика, свойства, применения / под ред. Левин Ю.И. Москва: Государственное издательство физико-математической литературы, 1960. 293 с.
11. Lockwood E.H. A book of curves. Cambridge University Press, 1961.
12. Комплексные числа и геометрические узоры [Электронный ресурс]. URL: <https://habr.com/p/565382/> (дата обращения: 07.02.2025).
13. Гильоши [Электронный ресурс]. URL: <https://habr.com/p/305354/> (дата обращения: 07.02.2025).
14. Роджерс Д., Адамс А. Математические основы машинной графики / под ред. Баяковский Ю.М., Галактионова В.А., Мартынюк В.В.; пер. Монахов П.А., Олохтонова Г.В., Волков Д.В. Москва: Мир, 2001. 604 с.